UNIVERSIDADE PAULISTA

ANDRÉ COSTA SAMPAIO - R0711E0

BEATRIZ ANDRADE VEZALLI - R186JJ8

FERNANDA NUNES LOPES - R1070E7

GIOVANNA MARINOSCKI DE PAULA BUENO - R0472H3

LEONARDO AFONSO FEITOSA DIAS - G9373H0

RYAN GABRIEL ALMEIDA DIAS - F359654

IDENTIFICAÇÃO E AUTENTICAÇÃO BIOMÉTRICA FACIAL APLICADA À PROTEÇÃO DE INFORMAÇÕES SENSÍVEIS

SÃO PAULO

2026

1 RELATÓRIO COM AS LINHAS DE CÓDIGO

1.1 ARQUITETURA E ESTRUTURA

O programa foi organizado em camadas, e cada camada corresponde a um ou mais arquivos Python. A interface gráfica (gui.py), construída com CustomTkinter e aberta pelo main.py, fica no topo; abaixo dela estão os módulos de fluxo (cadastro, autenticação e relatórios) e, em seguida, os serviços de processamento de imagens e de acesso ao banco de dados. Na base está o arquivo de configuração, usado por todos. Três princípios guiam o projeto: a separação de responsabilidades, em que cada arquivo cuida de um assunto; o baixo acoplamento, pois o módulo de processamento não conhece o banco e o banco não conhece a câmera, o que permite trocar o detector HOG por outro, ou o SQLite pelo MySQL, sem alterar o restante; e a configuração centralizada, que reúne valores como tolerância e tempo limite em um só lugar. O Quadro 1 resume os arquivos. Os trechos de código e os quadros deste relatório foram elaborados pelos autores (2026).

Quadro 1 – Módulos do programa e suas dependências

Arquivo	**Responsabilidade**	**Depende de**
main.py	Ponto de entrada: cria e executa a janela principal.	gui
gui.py	Interface gráfica: login, autenticação, cadastro, usuários, rastreio e arquivos.	todos os demais
config.py	Parâmetros globais do sistema.	nenhum
banco.py	Persistência: usuários, vetores, acessos, arquivos e logs.	config
processamento.py	Cinco fases do processamento de imagens e prova de vida.	config

cadastro.py	Cadastro de usuário pela webcam (janela do OpenCV).	banco, config, processamento
acesso.py	Autenticação em tempo real e regra de permissão por níveis.	banco, config, processamento
relatorio.py	Relatório de acessos (CSV e resumo) e imagens das fases.	banco, config, processamento

Fonte: elaborado pelos autores (2026).

```
APS/
├── main.py   gui.py   config.py   banco.py   processamento.py
├── cadastro.py   acesso.py   relatorio.py
├── requirements.txt   Reconhecimento.bat
├── cofre.db          (tabelas de arquivos e de logs do sistema)
├── dados/cofre.db    (usuários, vetores e acessos)
└── relatorios/       (CSV, resumos e fases/)
```

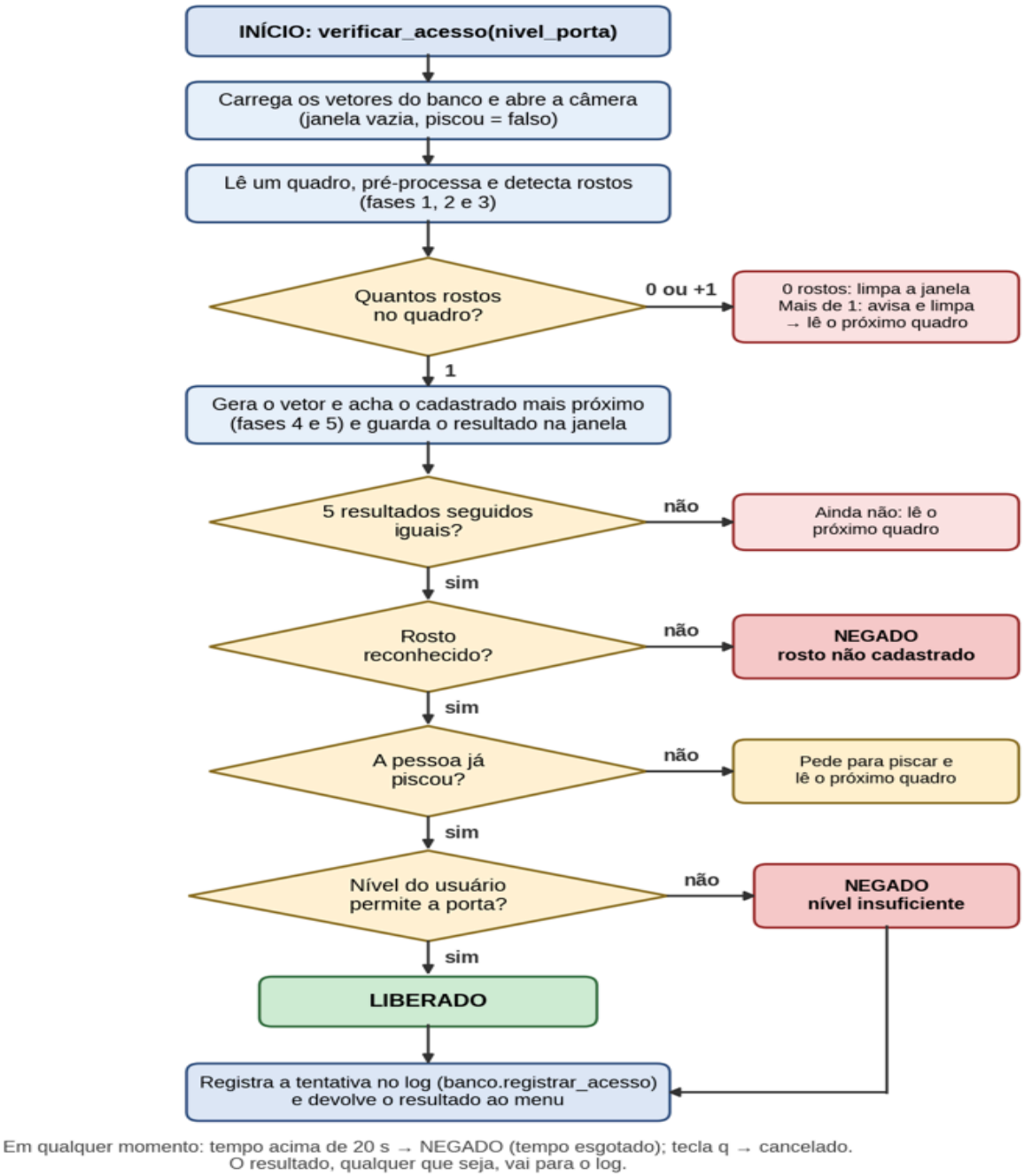
1.2 LÓGICA GERAL E FLUXO DE AUTENTICAÇÃO

A lógica do programa se dá em cinco passos:

- cadastro: a webcam captura 10 amostras do rosto, e cada uma é transformada em uma lista de 128 números (vetor), uma “impressão digital” numérica, gravada no banco com nome, cargo e nível de acesso;
- autenticação: ao tentar abrir o cofre, o rosto capturado também se torna um vetor de 128 números;
- comparação: o sistema mede a distância até os vetores cadastrados; se a menor distância for menor ou igual a 0,5, a pessoa é reconhecida, caso contrário é desconhecida;
- decisão: o acesso só é liberado se o mesmo resultado se repetir em cinco quadros seguidos, se a pessoa piscar (prova de vida) e se o nível dela for suficiente para a porta pedida;
- registro: toda tentativa, autorizada ou não, é gravada no banco e pode ser consultada nos relatórios.

A Figura 1 ilustra o que a função verificar_acesso faz a cada quadro de vídeo. O laço se repete até que uma decisão seja tomada, o tempo se esgote ou o operador cancele. A mesma lógica é reproduzida na interface gráfica (seção 8.4).

Figura 1 – Fluxo de autenticação da função verificar_acesso



Fonte: elaborado pelos autores (2026).

1.3 EXECUÇÃO DO PROGRAMA

Ao executar “python main.py” (ou Reconhecimento.bat), cria-se a janela CofreApp, definida em gui.py, que chama “banco.inicializar()” para criar as tabelas, caso ainda não existam, e exibe a tela de login. Após o login, o usuário navega entre as telas disponíveis para o seu perfil, e o programa termina ao fechar a janela. Nas primeiras versões, o main.py continha um menu em terminal, substituído pela interface gráfica; os módulos de cadastro, acesso e relatório continuam independentes dela.

1.4 MÓDULO CONFIG.PY

O módulo config.py não possui lógica, apenas guarda valores que os demais módulos leem por nome, como “config.TOLERANCIA”, funcionando como painel do sistema. O primeiro bloco define as pastas do banco e dos relatórios a partir de “Path(__file__).resolve().parent”, a raiz do projeto, o que faz os caminhos funcionarem em qualquer computador, sem endereços fixos como “C:\Users”. O segundo reúne os parâmetros de operação (câmera, escala, detector, tolerância, quadros de confirmação, amostras, limiar do olho e tempo limite), o que permite calibrar o sistema nos testes sem alterar a lógica dos módulos.

```
from pathlib import Path
BASE_DIR = Path (__file__).resolve().parent
DADOS_DIR = BASE_DIR / "dados"
DB_PATH = DADOS_DIR / "cofre.db"
RELATORIOS_DIR = BASE_DIR / "relatorios"

CAMERA_INDEX = 0
ESCALA = 0.5
MODELO_DETECCAO = "hog"
TOLERANCIA = 0.5
FRAMES_CONFIRMACAO = 5
AMOSTRAS_CADASTRO = 10
INTERVALO_AMOSTRA_S = 0.6
EXIGIR_PISCADA = True
EAR_LIMIAR = 0.21
```

```
TEMPO_LIMITE_S= 20

NIVEIS = {
   1: "Permissao geral",
   2: "Diretor de divisao",
   3: "Ministro do meio ambiente",
}
```

1.5 MÓDULO PROCESSAMENTO.PY

É o núcleo do sistema: cada uma das cinco fases do processamento de imagens é uma função independente, que apenas recebe uma imagem (ou vetores) e devolve um resultado, sem acessar o banco nem o operador, podendo ser reaproveitada pelos módulos de cadastro, autenticação e relatório. As bibliotecas importadas são o OpenCV (cv2), que abre a câmera e opera imagens; o NumPy (np), que fornece matrizes e cálculos; a face_recognition, construída sobre a dlib, que fornece detecção, vetores e pontos de referência; e o config. Reaproveitar bibliotecas consolidadas evita reescrever algoritmos.

1.3.1 aquisição e pré-processamento

```
import cv2
import numpy as np
import face_recognition
import config

def adquirir_camera():
   cap = cv2.VideoCapture(config.CAMERA_INDEX)
   if not cap.isOpened():
       raise RuntimeError(
           "Nao foi possivel abrir a camera. Verifique se ela está
conectada."
       )
   return cap

def preprocessar(frame_bgr):
   pequeno = cv2.resize(frame_bgr, (0, 0), fx=config.ESCALA,
fy=config.ESCALA)
   lab = cv2.cvtColor(pequeno, cv2.COLOR_BGR2LAB)
```

```
    canal_l, canal_a, canal_b = cv2.split(lab)
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
    canal_l = clahe.apply(canal_l)
    melhorado = cv2.cvtColor(cv2.merge((canal_l, canal_a, canal_b)),
cv2.COLOR_LAB2BGR)
    rgb = cv2.cvtColor(melhorado, cv2.COLOR_BGR2RGB)
    return np.ascontiguousarray(rgb)
```

A função adquirir_camera abre a webcam com “cv2.VideoCapture(config.CAMERA_INDEX)” e a devolve pronta; se “cap.isOpened()” indicar falha, levanta um RuntimeError com mensagem clara. O comando “cap.read()” devolve (ok, frame), em que frame é uma matriz altura × largura × 3 canais (BGR), com valores de 0 a 255.

A função preprocessar recebe o quadro bruto e devolve uma imagem menor, com melhor contraste e no formato exigido pela biblioteca. O “cv2.resize” reduz o quadro pela ESCALA, o que acelera as fases seguintes; a imagem vai para o espaço LAB, que separa o brilho (canal L) das cores; o CLAHE, equalizador adaptativo de contraste com grade 8 × 8 e amplificação limitada, é aplicado apenas ao canal L, para não exagerar o ruído; o resultado volta para BGR e é convertido para RGB, formato esperado pela face_recognition. O “np.ascontiguousarray” garante a memória contínua exigida pela dlib.

1.3.2 segmentação e extração de características

```
def segmentar(rgb):
    return face_recognition.face_locations(rgb,
model=config.MODELO_DETECCAO)

def caixa_no_frame_original(local):
    return tuple(int(v / config.ESCALA) for v in local)

def extrair_caracteristicas(rgb, locais):
    return face_recognition.face_encodings(rgb, locais)
```

O “face_recognition.face_locations” aplica o detector definido em MODELO_DETECCAO (hog) e devolve uma tupla (top, right, bottom, left) por rosto, ou uma lista vazia. Como a detecção ocorre na imagem reduzida, caixa_no_frame_original divide as coordenadas por ESCALA e converte para inteiros, de modo que a detecção seja rápida e o desenho no quadro original, nítido.

Na extração, para cada rosto já localizado, a rede neural pré-treinada da dlib calcula um vetor de 128 números (float64), na mesma ordem dos rostos. Comparar vetores é mais confiável do que comparar fotos pixel a pixel, pois o vetor é uma representação compacta da identidade do rosto.

1.3.3 classificação

```
def classificar(encoding, matriz):
   if len(matriz) == 0:
       return None, float("inf")
   distancias = face_recognition.face_distance(matriz, encoding)
   i = int(np.argmin(distancias))
   if distancias[i] <= config.TOLERANCIA:
       return i, float(distancias[i])
   return None, float(distancias[i])
```

A função classificar devolve None e distância infinita se a matriz estiver vazia. Caso contrário, “face_distance” calcula a distância euclidiana até cada vetor cadastrado e “np.argmin” encontra o mais próximo; se a menor distância for menor ou igual a TOLERANCIA, devolve a posição e a distância, senão devolve None, indicando rosto desconhecido. A função devolve a posição na matriz, e não o id; a tradução cabe ao módulo de acesso, pela lista infos. O limiar é o que permite responder “desconhecido” para quem não está cadastrado.

1.3.4 prova de vida

```
def _dist(p, q):
```

```
   return float(np.linalg.norm(np.array(p) - np.array(q)))


def razao_aspecto_olho(olho):
   return (_dist(olho[1], olho[5]) + _dist(olho[2], olho[4])) / (2.0 *
_dist(olho[0], olho[3]))

def ear_medio(rgb, locais):
   pontos = face_recognition.face_landmarks(rgb, locais)
   if not pontos:
       return None
   esq = razao_aspecto_olho(pontos[0]["left_eye"])
   dir_ = razao_aspecto_olho(pontos[0]["right_eye"])
   return (esq + dir_) / 2.0
```

A razão de aspecto do olho (EAR) indica se o olho está aberto ou fechado: EAR = (|P2 − P6| + |P3 − P5|) / (2 · |P1 − P4|), calculada com os 6 pontos do contorno do olho. Como é uma proporção entre altura e largura, independe da distância até a câmera: com o olho aberto fica aproximadamente constante e, ao fechar, cai para perto de zero. A função ear_medio usa “face_landmarks” para obter os pontos de left_eye e right_eye do primeiro rosto (o sistema trabalha com um rosto por vez) e calcula a média, que reduz oscilações; devolve None se não houver pontos, e o módulo de acesso ignora o quadro.

1.4 MÓDULO BANCO.PY

É o único arquivo que utiliza comandos SQL; os demais módulos chamam as suas funções. O banco SQLite possui cinco tabelas (Quadro 2): as três primeiras são criadas por inicializar(), em dados/cofre.db, e as duas últimas, usadas pela interface gráfica, são criadas sob demanda (seção 4.5).

Quadro 2 – Tabelas do banco de dados

Tabela	**O que guarda**
usuarios	Nome (único), cargo e nível de acesso (1 a 3).

encodings	Vetores de 128 números de cada usuário (BLOB), com exclusão em cascata.
acessos	Log de tentativas: data e hora, nome, níveis, resultado e motivo.
arquivos	Arquivos do cofre, com nível mínimo de visualização.
ogs_sistema	Trilha de auditoria das ações realizadas na interface.

Fonte: elaborado pelos autores (2026).

1.4.1 conexão e criação das tabelas

```
@contextmanager
def _conexao():
   config.DADOS_DIR.mkdir(parents=True, exist_ok=True)
   con = sqlite3.connect(config.DB_PATH)
   con.execute("PRAGMA foreign_keys = ON")
   try:
       yield con
       con.commit()
   except Exception:
       con.rollback()
       raise
   finally:
       con.close()

def inicializar():
   with _conexao() as con:
       con.executescript(
           """
           CREATE TABLE IF NOT EXISTS usuarios (
               id    INTEGER PRIMARY KEY AUTOINCREMENT,
               nome  TEXT NOT NULL UNIQUE,
               cargo TEXT,
               nivel INTEGER NOT NULL CHECK (nivel IN (1, 2, 3))
           );
           CREATE TABLE IF NOT EXISTS encodings (
               id          INTEGER PRIMARY KEY AUTOINCREMENT,
               usuario_id  INTEGER NOT NULL
                           REFERENCES usuarios(id) ON DELETE CASCADE,
               vetor       BLOB NOT NULL
           );
           CREATE TABLE IF NOT EXISTS acessos (
               id            INTEGER PRIMARY KEY AUTOINCREMENT,
               data_hora     TEXT NOT NULL,
               nome          TEXT NOT NULL,
```

```
                nivel_usuario INTEGER NOT NULL,
                nivel_porta   INTEGER NOT NULL,
                resultado     TEXT NOT NULL,
                motivo        TEXT
            );
            """
        )
```

A função _conexao é um gerenciador de contexto (@contextmanager), usado com “with”. Ela cria a pasta dados, se necessário, abre o banco e ativa “PRAGMA foreign_keys = ON”, que o SQLite deixa desligado a cada conexão e sem o qual a remoção em cascata não funcionaria. Se o bloco termina sem erro, confirma (commit); se ocorre uma exceção, desfaz (rollback) e repassa o erro; o “finally” sempre fecha a conexão. Isso garante a atomicidade: ou todas as operações de um bloco são gravadas, ou nenhuma.

A função inicializar cria as tabelas com “CREATE TABLE IF NOT EXISTS”, podendo ser chamada em toda execução. Em usuarios, o nome é obrigatório e único e o nivel só aceita 1, 2 ou 3 (CHECK); em encodings, usuario_id é chave estrangeira com ON DELETE CASCADE e o vetor é um BLOB. As restrições são uma defesa em profundidade: mesmo que a validação do programa falhe, o banco recusa níveis inválidos e nomes repetidos. A tabela acessos não tem chave estrangeira de propósito, pois o log precisa preservar o histórico mesmo que o usuário seja removido e registrar tentativas de pessoas desconhecidas.

1.4.2 usuários

```
def cadastrar_usuario(nome, cargo, nivel, encodings):
    """Grava o usuário + seus encodings e retorna o id."""
    if nivel not in config.NIVEIS:
        raise ValueError("Nível inválido.")
    if not encodings:
        raise ValueError("Não tem amostra de rosto para salvar.")
    try:
        with _conexao() as con:
            cur = con.execute(
```

```
                "INSERT INTO usuarios (nome, cargo, nivel) VALUES (?, ?,
?)",
                (nome, cargo, nivel),
            )
            uid = cur.lastrowid
            con.executemany(
                "INSERT INTO encodings (usuario_id, vetor) VALUES (?, ?)",
                [(uid, np.asarray(e, dtype=np.float64).tobytes()) for e in
encodings],
            )
            return uid
    except sqlite3.IntegrityError:
        raise ValueError(f"Já existe um usuário chamado '{nome}'.")

def listar_usuarios():
    with _conexao() as con:
        return con.execute(
            "SELECT u.id, u.nome, u.cargo, u.nivel, COUNT(e.id) "
            "FROM usuarios u LEFT JOIN encodings e ON e.usuario_id = u.id "
            "GROUP BY u.id ORDER BY u.id"
        ).fetchall()

def remover_usuario(uid):
    with _conexao() as con:
        cur = con.execute("DELETE FROM usuarios WHERE id = ?", (uid,))
        return cur.rowcount > 0
```

A função cadastrar_usuario valida as entradas (nível fora de config.NIVEIS ou lista de amostras vazia geram ValueError), grava o usuário e todos os vetores em uma única transação e devolve o id gerado. Os valores entram pelo símbolo “?”, em consulta parametrizada, defesa padrão contra injeção de SQL; “cur.lastrowid” liga os vetores ao usuário, e “executemany” grava todos de uma vez, cada vetor convertido para float64 e depois para bytes (“tobytes()”). Nome repetido gera IntegrityError, convertido em ValueError, e uma falha em qualquer passo desfaz tudo, de modo que não existam usuários sem vetores.

A função listar_usuarios usa LEFT JOIN e GROUP BY para devolver cada usuário com a quantidade de vetores, inclusive zero, o que revela cadastros incompletos. A função remover_usuario apaga por id (a vírgula em “(uid,)” é necessária para formar uma tupla) e, por cascata, os vetores, devolvendo verdadeiro se alguma linha foi removida (“cur.rowcount > 0”).

1.4.3 base de reconhecimento

```
def carregar_base():
   """Retorna (infos, matriz): infos[i] descreve a linha i da matriz de
encodings."""
   with _conexao() as con:
       linhas = con.execute(
           "SELECT u.id, u.nome, u.nivel, e.vetor "
           "FROM usuarios u JOIN encodings e ON e.usuario_id = u.id"
       ).fetchall()

   infos = [{"id": linha[0], "nome": linha[1], "nivel": linha[2]} for linha
in linhas]
   if linhas:
       matriz = np.array([np.frombuffer(linha[3], dtype=np.float64) for
linha in linhas])
   else:
       matriz = np.empty((0, 128))
   return infos, matriz
```

A função carregar_base lê todos os vetores e os entrega em duas estruturas alinhadas: infos, lista de dicionários com id, nome e nível, e matriz, matriz NumPy em que cada linha é um vetor, reconstruído por “np.frombuffer(..., dtype=np.float64)”, a conversão inversa do cadastro. A posição i de infos descreve o dono da linha i da matriz, e é assim que o módulo de acesso, a partir da posição devolvida por classificar, descobre quem é a pessoa. Com o banco vazio, devolve uma matriz 0 × 128, evitando tratamento especial.

1.4.4 log de acesso

```
def registrar_acesso(nome, nivel_usuario, nivel_porta, resultado,
motivo=""):
   with _conexao() as con:
       con.execute(
           "INSERT INTO acessos (data_hora, nome, nivel_usuario,
nivel_porta, resultado, motivo) "
           "VALUES (?, ?, ?, ?, ?, ?)",
           (datetime.now().isoformat(sep=" ", timespec="seconds"),
```

```
            nome, nivel_usuario, nivel_porta, resultado, motivo),
        )

def listar_acessos():
    with _conexao() as con:
        return con.execute(
            "SELECT data_hora, nome, nivel_usuario, nivel_porta, resultado,
motivo "
            "FROM acessos ORDER BY id"
        ).fetchall()
```

A função registrar_acesso grava cada tentativa com data e hora (“datetime.now().isoformat(sep=" ", timespec="seconds")”), nome, níveis, resultado e motivo (opcional), formando a trilha de auditoria. A função listar_acessos devolve todas as tentativas em ordem de id (cronológica), usadas pelo relatorio.py para gerar o CSV e as estatísticas; separar a consulta da apresentação mantém uma responsabilidade por módulo.

1.4.5 arquivo do cofre e logs do sistema

```
def inicializar_tabela_arquivos():
    conn = sqlite3.connect("cofre.db")
    cursor = conn.cursor()
    cursor.execute("""
        create table if not exists arquivos (
            id integer primary key autoincrement,
            nome_arquivo text,
            caminho text,
            nivel_minimo integer,
            data_upload text
        )
    """)
    conn.commit()
    conn.close()

def salvar_arquivo_banco(nome, caminho, nivel):
    import datetime
    inicializar_tabela_arquivos()
    conn = sqlite3.connect("cofre.db")
    cursor = conn.cursor()
    data_atual = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    cursor.execute(
```

```
        "insert into arquivos (nome_arquivo, caminho, nivel_minimo,
data_upload) values (?, ?, ?, ?)",
        (nome, caminho, nivel, data_atual)
    )
    conn.commit()
    conn.close()

def listar_arquivos_por_nivel(nivel_usuario):
    inicializar_tabela_arquivos()
    conn = sqlite3.connect("cofre.db")
    cursor = conn.cursor()
    # Mostra apenas arquivos cujo nivel_minimo seja menor ou igual ao nível
do utilizador logado
    cursor.execute(
        "select id, nome_arquivo, caminho, nivel_minimo, data_upload from
arquivos where nivel_minimo <= ? order by id desc",
        (nivel_usuario,)
    )
    resultados = cursor.fetchall()
    conn.close()
    return resultados

def remover_arquivo_banco(uid):
    inicializar_tabela_arquivos()
    conn = sqlite3.connect("cofre.db")
    cursor = conn.cursor()
    cursor.execute("delete from arquivos where id = ?", (uid,))
    conn.commit()
    conn.close()

def inicializar_tabela_logs():
    conn = sqlite3.connect("cofre.db")
    cursor = conn.cursor()
    cursor.execute("""
        create table if not exists logs_sistema (
            id integer primary key autoincrement,
            usuario text,
            acao text,
            detalhes text,
            data_hora text
        )
    """)
    conn.commit()
    conn.close()

def registrar_log(usuario, acao, detalhes):
    import datetime
    inicializar_tabela_logs()
    conn = sqlite3.connect("cofre.db")
    cursor = conn.cursor()
```

```
    data_atual = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    cursor.execute(
        "insert into logs_sistema (usuario, acao, detalhes, data_hora)
values (?, ?, ?, ?)",
        (usuario, acao, detalhes, data_atual)
    )
    conn.commit()
    conn.close()

def listar_logs():
    inicializar_tabela_logs()
    conn = sqlite3.connect("cofre.db")
    cursor = conn.cursor()
    cursor.execute("select id, usuario, acao, detalhes, data_hora from
logs_sistema order by id desc")
    resultados = cursor.fetchall()
    conn.close()
    return resultados
```

Para a interface gráfica, o módulo ganhou duas famílias de funções, que abrem conexão direta com o arquivo “cofre.db” da pasta de execução e criam suas tabelas sob demanda. Na primeira, salvar_arquivo_banco grava nome, caminho original (e não uma cópia), nível mínimo e data; listar_arquivos_por_nivel aplica a regra de visibilidade, devolvendo apenas arquivos com nivel_minimo menor ou igual ao nível do usuário; e remover_arquivo_banco apaga só o registro. Na segunda, registrar_log grava usuário, ação e detalhes, com data e hora automáticas, e listar_logs devolve os registros do mais recente ao mais antigo, para a tela de rastreio.

1.5 MÓDULO CADASTRO.PY

O módulo cadastro.py implementa a inscrição de um novo usuário (enrollment). Usa as fases 1 a 4 e grava o resultado com o banco, coletando 10 vetores do mesmo rosto em momentos diferentes, para tornar o reconhecimento mais robusto a variações de pose e expressão.

1.5.1 preparação

```
def cadastrar_pela_camera(nome, cargo, nivel):
   cap = proc.adquirir_camera()
   amostras = []
   ultimo = 0.0

   try:
       cv2.namedWindow(NOME_JANELA)
       try:
           cv2.setWindowProperty(NOME_JANELA, cv2.WND_PROP_TOPMOST, 1)
       except cv2.error:
           pass

       for _ in range(5):
           cap.read()
```

A função abre a câmera (“proc.adquirir_camera()”), cria a lista amostras e a variável ultimo (instante da última amostra) e a janela com “cv2.namedWindow”, tentando deixá-la à frente das demais (“WND_PROP_TOPMOST”), dentro de um “try” que ignora versões do OpenCV sem esse recurso. O laço “for _ in range(5)” descarta os 5 primeiros quadros, pois câmeras de notebook costumam entregar imagens escuras ou vazias enquanto a exposição se ajusta.

1.5.2 laço principal e decisão pelo número de rostos

```
while len(amostras) < config.AMOSTRAS_CADASTRO:
   ok, frame = cap.read()
   if not ok or frame is None:
       raise RuntimeError("Falha ao ler imagem da câmera.")

   rgb = proc.preprocessar(frame)
   locais = proc.segmentar(rgb)

msg = f"Amostras: {len(amostras)}/{config.AMOSTRAS_CADASTRO} (q =
cancelar)"
cor = (0, 255, 255)

if len(locais) == 0:
   msg = "Nenhum rosto encontrado."
   cor = (0, 0, 255)
elif len(locais) > 1:
```

```
    msg = "Apenas UMA pessoa no cadastro"
    cor = (0, 0, 255)
else:
    top, right, bottom, left = proc.caixa_no_frame_original(locais[0])
    cv2.rectangle(frame, (left, top), (right, bottom), (0, 255, 0), 2)
    agora = time.time()
    if agora - ultimo >= config.INTERVALO_AMOSTRA_S:
        enc = proc.extrair_caracteristicas(rgb, locais)[0]
        amostras.append(enc)
        ultimo = agora
        print(f"  amostra {len(amostras)}/{config.AMOSTRAS_CADASTRO}
capturada")
```

O laço “while len(amostras) < config.AMOSTRAS_CADASTRO” termina sozinho quando as 10 amostras são coletadas. Cada volta lê um quadro (RuntimeError se falhar), pré-processa e segmenta. A mensagem mostra o progresso (por exemplo, “Amostras: 3/10”), com cores no padrão BGR: (0, 0, 255) vermelho, (0, 255, 0) verde e (0, 255, 255) amarelo. Sem rosto, ou com mais de um, mostra um aviso em vermelho e não coleta amostra, para não misturar duas pessoas no mesmo usuário. Com exatamente um rosto, desenha o retângulo verde e coleta a amostra.

1.5.3 exibição, tratamento de erros e gravação

```
cv2.putText(frame, msg, (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.7, cor, 2)
cv2.imshow(NOME_JANELA, frame)
if cv2.waitKey(1) & 0xFF == ord("q"):
    print("Cadastro cancelado.")
    return None

except Exception:
    # mostra o erro completo (arquivo, linha e motivo) em vez de esconder
    print("\n*** ERRO DURANTE A CAPTURA ***")
    traceback.print_exc()
    return None
finally:
    cap.release()
    cv2.destroyAllWindows()

# salvar no banco (fora do try acima para o erro do banco aparecer
separado)
```

```
try:
   uid = banco.cadastrar_usuario(nome, cargo, nivel, amostras)
except Exception:
   print("\n*** ERRO AO SALVAR NO BANCO ***")
   traceback.print_exc()
   return None

print(f"Usuário '{nome}' cadastrado (id {uid}, nível {nivel}) com
{len(amostras)} amostras.")
return uid
```

O “cv2.putText” escreve a mensagem no quadro e “cv2.imshow” o exibe; “cv2.waitKey(1)” aguarda 1 ms por uma tecla e dá ao OpenCV o tempo de redesenhar a janela, sem o que ela pareceria travada. A tecla “q” cancela o cadastro, e a função devolve None sem gravar nada. O “except” imprime o relato completo (“traceback.print_exc()”) e o “finally” sempre libera a câmera e fecha as janelas, evitando webcam ocupada. Terminada a captura, “banco.cadastrar_usuario” grava o usuário e os vetores em um segundo “try”, separado de propósito para que um erro do banco (como nome duplicado) não seja confundido com problema de câmera; em caso de sucesso, a função devolve o id do novo usuário.

1.6 MÓDULO ACESSO.PY

É o módulo mais importante para a segurança, pois decide liberar ou negar o acesso, integrando as cinco fases, a regra de permissão por nível, a prova de vida e o registro em log. Na interface gráfica, a TelaAutenticacao implementa o laço em uma thread, mas reaproveitar _pode_acessar e desenhar; a função verificar_acesso permanece disponível em modo console.

1.6.1 regra de permissão e desenho

```
def _pode_acessar(nivel_usuario, nivel_porta):
   if nivel_porta == 3:
       return nivel_usuario == 3
```

```
    return nivel_usuario >= nivel_porta

def desenhar(frame, local, texto, cor):
    top, right, bottom, left = proc.caixa_no_frame_original(local)
    cv2.rectangle(frame, (left, top), (right, bottom), cor, 2)
    cv2.putText(frame, texto, (left, max(bottom + 25, 25)),
                cv2.FONT_HERSHEY_SIMPLEX, 0.7, cor, 2)
```

A função _pode_acessar responde se um usuário pode abrir uma porta: a porta de nível 3 só admite usuário de nível 3 (“nivel_usuario == 3”); nas demais vale a regra hierárquica “nivel_usuario >= nivel_porta”. A função desenhar converte a caixa para o quadro original (“caixa_no_frame_original”), traça o retângulo (“cv2.rectangle”) e escreve o texto (“cv2.putText”); “max(bottom + 25, 25)” mantém o texto na área visível, e isolar o desenho evita repetir essas linhas.

1.6.2 preparação da função (verificar_acesso)

```
def verificar_acesso(nivel_porta):
    infos, matriz = banco.carregar_base()
    if len(infos) == 0:
        raise RuntimeError("Nenhum usuario cadastrado :// cadastre alguem
primeiro!")

    cap = proc.adquirir_camera()
    janela = deque(maxlen=config.FRAMES_CONFIRMACAO)
    piscou = not config.EXIGIR_PISCADA
    olho_fechado = False
    inicio = time.time()

    concedido, nome, nivel_usuario, motivo = False, "Desconhecido", 0,
"Tempo esgotado"
```

A função carrega os dados com “banco.carregar_base()” (e levanta um erro, antes de abrir a câmera, se não houver ninguém cadastrado), abre a câmera e define as variáveis de controle. A variável janela é um “deque” com capacidade FRAMES_CONFIRMACAO (5), que descarta o item mais antigo quando entra um novo: uma janela deslizante sobre os últimos reconhecimentos. A variável piscou

começa como “not config.EXIGIR_PISCADA”, olho_fechado indica se o olho já foi visto fechado e inicio guarda o horário de partida. O resultado padrão é pessimista (negado, Desconhecido, nível 0, Tempo esgotado), o princípio da falha segura: se o laço terminar sem decisão, na dúvida, o acesso é negado.

1.6.3 laço, reconhecimento e prova de vida

```
try:
   while True:
       if time.time() - inicio > config.TEMPO_LIMITE_S:
           break
       ok, frame = cap.read()
       if not ok:
           raise RuntimeError("falha ao ler imagem da camera !")
       rgb = proc.preprocessar(frame)
       locais = proc.segmentar(rgb)
       msg, cor = f"Aproxime o rosto (porta nivel {nivel_porta})", (0, 255,
255)

else:
   local = locais[0]
   enc = proc.extrair_caracteristicas(rgb, [local])[0]
   idx, dist = proc.classificar(enc, matriz)
   janela.append(idx if idx is None else infos[idx]["id"])

if config.EXIGIR_PISCADA and not piscou:
   ear = proc.ear_medio(rgb, [local])
   if ear is not None:
       if ear < config.EAR_LIMIAR:
           olho_fechado = True
       elif olho_fechado:
           piscou = True

desenhar(frame, local, "analisando...", (255, 200, 0))
```

O “while True” só termina por “break” (decisão, tempo esgotado ou cancelamento). A cada volta, se o tempo decorrido passou de TEMPO_LIMITE_S (20 s), o laço é interrompido, o que impede o sistema de esperar indefinidamente; depois lê o quadro e executa as fases 1 a 3. Com um único rosto, extrair_caracteristicas gera o vetor (fase 4) e classificar o compara com a matriz

(fase 5), devolvendo idx e a distância. A chamada “janela.append” guarda o id do usuário (“idx if idx is None else infos[idx]["id"]”), e não a posição, o que torna a comparação entre quadros independente da ordem dos vetores.

Enquanto a piscada é exigida e ainda não ocorreu, ear_medio calcula o EAR de cada quadro (ignorado se None). EAR abaixo de EAR_LIMIAR marca olho_fechado = True; EAR acima do limiar com o olho marcado como fechado completa o ciclo fecha-abre: piscou = True. Exigir o ciclo completo evita aceitar uma fotografia de olhos fechados; a limitação é que um vídeo em que a pessoa pisca ainda poderia enganar o mecanismo.

1.6.4 tomada de decisão, exibição e restrição

```
if len(janela) == janela.maxlen and len(set(janela)) == 1:
   if idx is None:
       nome, nivel_usuario = "Desconhecido", 0
       concedido, motivo = False, "Rosto não cadastrado"
       desenhar(frame, local, "NEGADO - desconhecido", (0, 0, 255))
       break
   if not piscou:
       msg, cor = "Pisque para confirmar que voce e real", (0, 165, 255)
   else:
       nome, nivel_usuario = infos[idx]["nome"], infos[idx]["nivel"]
       if _pode_acessar(nivel_usuario, nivel_porta):
           concedido, motivo = True, "Acesso autorizado"
           desenhar(frame, local, f"LIBERADO - {nome}", (0, 255, 0))
       else:
           concedido = False
           motivo = f"Nível {nivel_usuario} insuficiente para porta
{nivel_porta}"
           desenhar(frame, local, f"NEGADO - {nome} (nivel
{nivel_usuario})", (0, 0, 255))
       break

     cv2.putText(frame, msg, (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.7, cor,
2)
       cv2.imshow("Cofre - autenticacao (q = sair)", frame)
       if cv2.waitKey(1) & 0xFF == ord("q"):
           motivo = "Cancelado pelo usuário"
           break

   # mostra o resultado final por 2 segundos
```

```
   if motivo != "Cancelado pelo usuário" and motivo != "Tempo esgotado":
       cv2.imshow("Cofre - autenticacao (q = sair)", frame)
       cv2.waitKey(2000)
finally:
   cap.release()
   cv2.destroyAllWindows()

banco.registrar_acesso(nome, nivel_usuario, nivel_porta,
                       "CONCEDIDO" if concedido else "NEGADO", motivo)
return concedido, nome, motivo
```

A decisão exige que a janela esteja cheia (“len(janela) == janela.maxlen”) e que os 5 quadros concordem (“len(set(janela)) == 1”). Se o resultado é None, o rosto é desconhecido e o acesso é negado; se a pessoa foi reconhecida, mas não piscou, o sistema pede a piscada e o laço continua; se foi reconhecida e piscou, busca nome e nível em infos e chama _pode_acessar, encerrando o laço. Exigir 5 quadros iguais protege contra decisões baseadas em um único quadro ruim ou em alternância entre pessoas parecidas.

Ao fim de cada volta sem decisão, a função escreve a mensagem e exibe o quadro (“q” cancela). Quando há decisão, o último quadro fica 2 s na tela (“waitKey(2000)”). O “finally” libera a câmera e “banco.registrar_acesso” grava a tentativa, qualquer que seja o resultado, inclusive cancelamentos e tempo esgotado, e a função devolve (concedido, nome, motivo). Os textos CONCEDIDO, NEGADO e desconhecido são os mesmos que o relatório procura ao contar tentativas.

1.7 MÓDULO RELATORIO.PY

O módulo reúne duas funcionalidades extras: o relatório estatístico dos acessos e a gravação das imagens de cada fase, usadas para documentar o trabalho.

1.7.1 relatório de acessos

```
def gerar_relatorio_acessos():
   acessos = banco.listar_acessos()
   config.RELATORIOS_DIR.mkdir(parents=True, exist_ok=True)
   carimbo = datetime.now().strftime("%Y%m%d_%H%M%S")

   caminho_csv = config.RELATORIOS_DIR / f"acessos_{carimbo}.csv"
   with open(caminho_csv, "w", newline="", encoding="utf-8") as f:
       w = csv.writer(f)
       w.writerow(["data_hora", "nome", "nivel_usuario", "nivel_porta",
"resultado", "motivo"])
       w.writerows(acessos)

total = len(acessos)
concedidos = sum(1 for a in acessos if a[4] == "CONCEDIDO")
negados = total - concedidos
desconhecidos = sum(1 for a in acessos if a[1] == "Desconhecido")
por_usuario = Counter(a[1] for a in acessos if a[4] == "CONCEDIDO")

linhas = [
   "Relatório de acessos ao cofre",
   f"Gerado em: {datetime.now().strftime('%d/%m/%Y %H:%M:%S')}",
   f"Total de tentativas: {total}",
   f"Concedidos: {concedidos}",
   f"Negados: {negados} (rostos desconhecidos: {desconhecidos})",
   "Acessos concedidos por usuário:",
] + [f" - {n}: {q}" for n, q in por_usuario.most_common()]

caminho_txt = config.RELATORIOS_DIR / f"resumo_{carimbo}.txt"
caminho_txt.write_text("\n".join(linhas), encoding="utf-8")
print("\n".join(linhas))
print(f"\nArquivos salvos em: {config.RELATORIOS_DIR}")
```

A função gerar_relatorio_acessos lê o log (“banco.listar_acessos()”), garante a pasta (“mkdir(parents=True, exist_ok=True)”) e grava um CSV com data e hora no nome (por exemplo, 20261014_153210); “newline="" ” evita linhas em branco extras no Windows e “encoding="utf-8"” preserva os acentos. Em seguida, calcula o total de tentativas, os concedidos, os negados (total menos concedidos) e os desconhecidos, a partir de a[4] (resultado) e a[1] (nome); o “Counter” conta os acessos concedidos por usuário. O resumo é montado como lista de linhas (título, data, totais e uma linha por usuário, do mais ao menos frequente), gravado em resumo_<carimbo>.txt com “write_text” e também exibido na tela.

1.7.2 imagens de cada fase

```
def salvar_fases():
   cap = proc.adquirir_camera()
   try:
       for _ in range(10):
           ok, frame = cap.read()
       if not ok:
           raise RuntimeError("Falha ao ler imagem da câmera!")
   finally:
       cap.release()

   pasta = config.RELATORIOS_DIR / "fases"
   pasta.mkdir(parents=True, exist_ok=True)

   cv2.imwrite(str(pasta / "1_aquisicao.jpg"), frame)
   rgb = proc.preprocessar(frame)
   cv2.imwrite(str(pasta / "2_preprocessamento.jpg"), cv2.cvtColor(rgb,
cv2.COLOR_RGB2BGR))

   locais = proc.segmentar(rgb)
   seg = cv2.cvtColor(rgb, cv2.COLOR_RGB2BGR)
   for top, right, bottom, left in locais:
       cv2.rectangle(seg, (left, top), (right, bottom), (0, 255, 0), 2)
   cv2.imwrite(str(pasta / "3_segmentacao.jpg"), seg)

   if locais:
       enc = proc.extrair_caracteristicas(rgb, locais)[0]
       (pasta / "4_caracteristicas.txt").write_text(
           "Vetor de 128 características do rosto:\n" + ",
".join(f"{v:.4f}" for v in enc),
           encoding="utf-8",
       )

   print(f"Imagens das fases salvas em: {pasta} (rostos detectados:
{len(locais)})")
```

A função salvar_fases tira uma foto e salva a imagem de cada fase: lê 10 quadros e mantém o último, pois os primeiros podem estar escuros; salva a imagem bruta (fase 1) e a pré-processada (fase 2, convertida de RGB para BGR porque “cv2.imwrite” espera essa ordem); desenha um retângulo verde sobre cada rosto (fase 3) e, se houver rosto, salva em texto os 128 valores do vetor do primeiro, com

4 casas decimais (fase 4). A fase 5 produz uma decisão, documentada pelo log e pelo relatório.

1.8 INTERFACE GRÁFICA (GUI.PY)

1.8.1 visão geral

O módulo gui.py reúne a interface, desenvolvida com CustomTkinter, Pillow (para exibir os quadros da câmera) e OpenCV (para a captura). Ela não contém regras de reconhecimento: coleta dados, chama as funções dos outros módulos e mostra os resultados. Define a janela principal (CofreApp) e seis telas (Quadro 3), cada uma representada por uma classe derivada de CTkFrame. As duas últimas linhas das importações definem o tema escuro e a paleta azul.

Quadro 9 – Classes da interface gráfica

Classe	**Função**	**Quem acessa**
CofreApp	Janela principal: sessão, menu lateral e troca de telas.	Todos
TelaLogin	Login por identificação e senha ou por reconhecimento facial.	Todos
TelaAutenticacao	Abertura do cofre por nível de porta, com prova de vida.	Todos
TelaCadastro	Cadastro de usuários com captura de amostras do rosto.	ROOT
TelaUsuarios	Listagem e exclusão de usuários.	ROOT (exclusão)
TelaRastreio	Consulta e exportação dos logs de auditoria.	ROOT

TelaRelatorios	Envio, listagem, abertura e exclusão de arquivos por nível.	Todos (envio e exclusão: ROOT)

Fonte: elaborado pelos autores (2026).

```
import os
import time
import tkinter as tk
from tkinter import messagebox, ttk, filedialog
import customtkinter as ctk
import cv2
import numpy as np
from PIL import Image, ImageTk

import acesso
import banco
import cadastro
import config
import processamento as proc
import relatorio

ctk.set_appearance_mode("Dark")
ctk.set_default_color_theme("blue")
```

1.8.2 janela principal e menu

```
class CofreApp(ctk.CTk):

   def __init__(self):
       super().__init__()

       self.title("Sistema de Cofre Biométrico")
       self.geometry("1050x680")
       self.resizable(False, False)

       banco.inicializar()

       self.grid_columnconfigure(0, weight=1)
       self.grid_rowconfigure(0, weight=1)

       self.container_principal = ctk.CTkFrame(self,
fg_color="transparent")
       self.container_principal.grid(row=0, column=0, sticky="nsew")
       self.container_principal.grid_columnconfigure(0, weight=1)
       self.container_principal.grid_rowconfigure(0, weight=1)
```

```
        self.usuario_logado = None
        self.is_root = False
        self.nivel_usuario = 3

        self.mostrar_tela_login()

    def limpar_container(self):
        for widget in self.winfo_children():
            widget.destroy()

    def mostrar_tela_login(self):
        self.limpar_container()
        self.usuario_logado = None
        self.is_root = False

        self.grid_columnconfigure(0, weight=1)
        self.grid_columnconfigure(1, weight=0)

        self.container_principal = ctk.CTkFrame(self,
fg_color="transparent")
        self.container_principal.grid(row=0, column=0, sticky="nsew")
        self.container_principal.grid_columnconfigure(0, weight=1)
        self.container_principal.grid_rowconfigure(0, weight=1)

        tela_login = TelaLogin(self.container_principal, self)
        tela_login.grid(row=0, column=0, sticky="nsew")

    def iniciar_sistema(self, nome_usuario, is_root=False, nivel=3):
        self.usuario_logado = nome_usuario
        self.is_root = is_root
        self.nivel_usuario = nivel
        self.limpar_container()

        self.grid_columnconfigure(0, weight=0)
        self.grid_columnconfigure(1, weight=1)

        self._criar_menu_lateral()

        self.container = ctk.CTkFrame(self)
        self.container.grid(row=0, column=1, sticky="nsew", padx=10,
pady=10)
        self.container.grid_columnconfigure(0, weight=1)
        self.container.grid_rowconfigure(0, weight=1)

        self.telas = {}
        self._inicializar_telas()

        self.mostrar_tela("Autenticação")
```

```
        if self.is_root:
            btn_cadastro = ctk.CTkButton(
                menu_frame,
                text="👤 Cadastrar Utilizador",
                height=35,
                fg_color="#2b7a0b",
                hover_color="#1e5407",
                command=lambda: self.mostrar_tela("Cadastro"),
            )
            btn_cadastro.pack(padx=20, pady=6)

            btn_usuarios = ctk.CTkButton(
                menu_frame,
                text="📋 Utilizadores",
                height=35,
                command=lambda: self.mostrar_tela("Usuarios"),
            )
            btn_usuarios.pack(padx=20, pady=6)

    def _inicializar_telas(self):
        self.telas["Autenticação"] = TelaAutenticacao(self.container, self)
        if self.is_root:
            self.telas["Cadastro"] = TelaCadastro(self.container, self)
            self.telas["Usuarios"] = TelaUsuarios(self.container, self)
            self.telas["Rastreio"] = TelaRastreio(self.container, self)

        self.telas["Relatorios"] = TelaRelatorios(self.container, self)

        for tela in self.telas.values():
            tela.grid(row=0, column=0, sticky="nsew")

    def mostrar_tela(self, nome_tela):
        for nome, tela in self.telas.items():
            if hasattr(tela, "parar_camera"):
                tela.parar_camera()

        if nome_tela in self.telas:
            tela = self.telas[nome_tela]
            tela.tkraise()
            if hasattr(tela, "ao_exibir"):
                tela.ao_exibir()
```

A classe CofreApp herda de “ctk.CTk”. O construtor define título, tamanho fixo (1050 × 680), chama “banco.inicializar()” e cria o contêiner e os atributos da sessão (usuario_logado, is_root e nivel_usuario). O método limpar_container destrói os componentes da janela, e mostrar_tela_login zera a sessão antes de exibir a

TelaLogin, o que implementa o logout. Após um login bem-sucedido, iniciar_sistema grava a sessão, divide a janela em duas colunas (menu lateral e conteúdo), cria as telas e abre a autenticação.

No menu lateral, a condição “if self.is_root” controla quais botões aparecem: somente o administrador vê Cadastrar Utilizador, Utilizadores e Rastreio e Logs. O método _inicializar_telas cria as telas de cadastro, usuários e rastreio apenas para o ROOT, de modo que um usuário comum não tenha como acessá-las, pois elas nem existem na sessão dele. Todas ficam empilhadas na mesma célula da grade; mostrar_tela chama parar_camera nas telas que a possuem, para que a webcam nunca fique ligada em tela invisível, traz a escolhida com “tkraise()” e chama ao_exibir, se existir, para atualizar seus dados.

1.8.3 tela de login

```
    def login_senha(self):
        user = self.ent_user.get().strip()
        senha = self.ent_pass.get().strip()

        if user.upper() == "ROOT" and senha == "1234":
            banco.registrar_log("ROOT", "LOGIN", "Login efetuado com ID e
Senha (Admin)")
            messagebox.showinfo("Sucesso", "Login efetuado como ROOT
(Administrador)!")
            self.controller.iniciar_sistema("ROOT (Admin)", is_root=True,
nivel=3)
            return

        try:
            infos, matriz = banco.carregar_base()
            encontrado = None
            for u in infos:
                if u["nome"].lower() == user.lower():
                    encontrado = u
                    break
            if encontrado and senha == "1234":
                banco.registrar_log(encontrado['nome'], "LOGIN", f"Login
efetuado com senha (Nível {encontrado['nivel']})")
                messagebox.showinfo("Sucesso", f"Login efetuado como
{encontrado['nome']}!")
```

```
                self.controller.iniciar_sistema(encontrado['nome'],
is_root=False, nivel=encontrado['nivel'])
                return
        except Exception:
            pass

        messagebox.showerror("Erro de Login", "Credenciais inválidas!
Verifique o utilizador e a senha.")

    def login_com_rosto(self):
        top = ctk.CTkToplevel(self)
        top.title("Login Biométrico Facial")
        top.geometry("600x500")
        top.grab_set()

        lbl_video = ctk.CTkLabel(top, text="A iniciar câmara...", width=480,
height=320, fg_color="black")
        lbl_video.pack(pady=20)

        lbl_msg = ctk.CTkLabel(top, text="Aproxime o rosto para se
identificar...", font=ctk.CTkFont(size=14))
        lbl_msg.pack(pady=5)

        def processar_rosto():
            try:
                infos, matriz = banco.carregar_base()
                if len(infos) == 0:
                    lbl_msg.configure(text="Nenhum utilizador cadastrado na
base!", text_color="red")
                    return
            except Exception as e:
                lbl_msg.configure(text=f"Erro: {str(e)}", text_color="red")
                return

            cap = proc.adquirir_camera()
            from collections import deque
            janela = deque(maxlen=config.FRAMES_CONFIRMACAO)
            inicio = time.time()
            sucesso = False
            user_encontrado = None

            try:
                while top.winfo_exists():
                    if time.time() - inicio > config.TEMPO_LIMITE_S:
                        break
                    ok, frame = cap.read()
                    if not ok:
                        break

                    rgb = proc.preprocessar(frame)
```

```
                    locais = proc.segmentar(rgb)

                    if len(locais) == 1:
                        local = locais[0]
                        enc = proc.extrair_caracteristicas(rgb, [local])[0]
                        idx, dist = proc.classificar(enc, matriz)
                        janela.append(idx if idx is None else
infos[idx]["id"])

                        if len(janela) == janela.maxlen and len(set(janela))
== 1:
                            if idx is not None:
                                user_encontrado = infos[idx]
                                sucesso = True
                                break

                    img_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
                    img_pil = Image.fromarray(img_rgb)
                    img_tk = ImageTk.PhotoImage(image=img_pil)
                    if top.winfo_exists():
                        lbl_video.configure(image=img_tk, text="")
                        lbl_video.image = img_tk
                    time.sleep(0.03)
            finally:
                cap.release()

            if top.winfo_exists():
                top.destroy()

            if sucesso:
                nome = user_encontrado['nome']
                nivel = user_encontrado['nivel']
                banco.registrar_log(nome, "LOGIN_FACIAL", f"Login facial
bem-sucedido (Nível {nivel})")
                messagebox.showinfo("Sucesso", f"Bem-vindo, {nome}!")
                is_root_user = (nome.upper() == "ROOT")
                self.controller.iniciar_sistema(nome, is_root=is_root_user,
nivel=3 if is_root_user else nivel)
            else:
                messagebox.showwarning("Falha", "Rosto não reconhecido ou
tempo esgotado.")

        import threading
        threading.Thread(target=processar_rosto, daemon=True).start()
```

A TelaLogin oferece dois caminhos. No login por identificação e senha, o ROOT (sem diferenciar maiúsculas) com a senha definida no sistema entra como administrador, com nível 3; os demais usuários são procurados na base e entram

com o nível cadastrado. A senha é uma constante (“1234”), simplificação adotada na demonstração; em um sistema real seria individual e armazenada de forma protegida (hash).

No login facial, uma janela modal (“CTkToplevel” com “grab_set()”) mostra o vídeo, e a função processar_rosto, executada em thread (“threading.Thread(..., daemon=True)”) para não congelar a interface, repete a lógica de reconhecimento: exatamente um rosto, classificação e confirmação em 5 quadros iguais dentro do tempo limite. Cada quadro é convertido de BGR para RGB e depois em imagem do Tkinter (“ImageTk.PhotoImage”); a referência “lbl_video.image = img_tk” impede que o Python descarte a imagem. O login facial não exige a piscada, reservada à abertura do cofre; ao final, registra LOGIN_FACIAL no log e inicia a sessão (nível 3 se for o ROOT).

1.8.4 tela de autenticação

```
    def iniciar_autenticacao(self):
        if self.rodando:
            return

        nivel_porta = int(self.combo_nivel.get().split(" ")[0])
        try:
            infos, matriz = banco.carregar_base()
            if len(infos) == 0:
                messagebox.showwarning("Aviso", "Nenhum utilizador
cadastrado na base!")
                return
        except Exception as e:
            messagebox.showerror("Erro no Banco", str(e))
            return

        self.rodando = True
        self.btn_iniciar.configure(state="disabled")

        import threading
        threading.Thread(
            target=self._loop_autenticacao, args=(nivel_porta, infos,
matriz), daemon=True
        ).start()
```

```
   def _loop_autenticacao(self, nivel_porta, infos, matriz):
       from collections import deque

       self.cap = proc.adquirir_camera()
       janela = deque(maxlen=config.FRAMES_CONFIRMACAO)
       piscou = not config.EXIGIR_PISCADA
       olho_fechado = False
       inicio = time.time()

       concedido, nome, nivel_usuario, motivo = (
           False,
           "Desconhecido",
           0,
           "Tempo esgotado",
       )

       try:
           while self.rodando:
               if time.time() - inicio > config.TEMPO_LIMITE_S:
                   break
               ok, frame = self.cap.read()
               if not ok:
                   break

               rgb = proc.preprocessar(frame)
               locais = proc.segmentar(rgb)

               if len(locais) == 0:
                   janela.clear()
                   self.lbl_status.configure(
                       text=f"Aproxime o rosto (Porta nível
{nivel_porta})",
                       text_color="yellow",
                   )
               elif len(locais) > 1:
                   janela.clear()
                   self.lbl_status.configure(
                       text="Apenas UMA pessoa de cada vez!",
text_color="red"
                   )
                   for rosto in locais:
                       acesso.desenhar(frame, rosto, "", (0, 0, 255))
               else:
                   local = locais[0]
                   enc = proc.extrair_caracteristicas(rgb, [local])[0]
                   idx, dist = proc.classificar(enc, matriz)
                   janela.append(idx if idx is None else infos[idx]["id"])

                   if config.EXIGIR_PISCADA and not piscou:
```

```
                        ear = proc.ear_medio(rgb, [local])
                        if ear is not None:
                            if ear < config.EAR_LIMIAR:
                                olho_fechado = True
                            elif olho_fechado:
                                piscou = True

                    acesso.desenhar(frame, local, "A analisar...", (255,
200, 0))

                    if len(janela) == janela.maxlen and len(set(janela)) ==
1:
                        if idx is None:
                            nome, nivel_usuario = "Desconhecido", 0
                            concedido, motivo = False, "Rosto não
cadastrado"
                            acesso.desenhar(frame, local, "NEGADO", (0, 0,
255))
                            break
                        if not piscou:
                            self.lbl_status.configure(
                                text="Pisque para confirmar que é real!",
                                text_color="orange",
                            )
                        else:
                            nome, nivel_usuario = (
                                infos[idx]["nome"],
                                infos[idx]["nivel"],
                            )
                            if acesso._pode_acessar(nivel_usuario,
nivel_porta):
                                concedido, motivo = True, "Acesso
autorizado"
                                acesso.desenhar(
                                    frame, local, f"LIBERADO - {nome}", (0,
255, 0)
                                )
                            else:
                                concedido = False
                                motivo = f"Nível {nivel_usuario}
insuficiente para porta {nivel_porta}"
                                acesso.desenhar(
                                    frame, local, f"NEGADO - {nome}", (0, 0,
255)
                                )
                            break

                self._atualizar_video(frame)
                time.sleep(0.03)
```

```
        finally:
            if self.cap:
                self.cap.release()
            self.rodando = False
            self.btn_iniciar.configure(state="normal")
            self.after(0, self.resetar_visor)

        banco.registrar_acesso(
            nome,
            nivel_usuario,
            nivel_porta,
            "CONCEDIDO" if concedido else "NEGADO",
            motivo,
        )

        # Regista log de auditoria da tentativa de acesso ao cofre
        banco.registrar_log(self.controller.usuario_logado,
"AUTENTICACAO_COFRE", f"Porta Nível {nivel_porta} - Utilizador: {nome} -
Status: {'CONCEDIDO' if concedido else 'NEGADO'}")

        if concedido:
            self.lbl_status.configure(
                text=f"✅ ACESSO LIBERADO: {nome}", text_color="green"
            )
        else:
            self.lbl_status.configure(
                text=f"❌ ACESSO NEGADO: {motivo}", text_color="red"
            )
```

A TelaAutenticacao exibe o vídeo (600 × 360), o status, a escolha do nível da porta (1 – Geral, 2 – Diretor, 3 – Ministro) e o botão Iniciar Leitura. O método iniciar_autenticacao ignora cliques repetidos, extrai o nível da porta do texto escolhido, carrega a base (avisando se estiver vazia), desabilita o botão e inicia _loop_autenticacao em thread. O laço repete o de verificar_acesso (seção 6): trata zero, mais de um e exatamente um rosto, aplica a prova de vida e a confirmação em 5 quadros e decide com “acesso._pode_acessar”, usando “acesso.desenhar” para marcar o rosto. O “finally” libera a câmera, reabilita o botão e reinicia o visor; a tentativa é gravada na tabela acessos (“banco.registrar_acesso”) e no log de auditoria (AUTENTICACAO_COFRE), e o status final mostra ACESSO LIBERADO (verde) ou ACESSO NEGADO, com o motivo (vermelho).

1.8.5 tela de cadastro

```
    def iniciar_captura(self):
        nome = self.ent_nome.get().strip()
        cargo = self.ent_cargo.get().strip()
        nivel = int(self.combo_nivel.get().split(" ")[0])

        if not nome:
            messagebox.showwarning("Aviso", "Digite o nome completo.")
            return

        self.rodando = True
        self.btn_capturar.configure(state="disabled")

        import threading
        threading.Thread(
            target=self._loop_captura, args=(nome, cargo, nivel),
daemon=True
        ).start()

    def _loop_captura(self, nome, cargo, nivel):
        self.cap = proc.adquirir_camera()
        amostras = []
        ultimo = 0.0

        try:
            while self.rodando and len(amostras) < config.AMOSTRAS_CADASTRO:
                ok, frame = self.cap.read()
                if not ok:
                    break

                rgb = proc.preprocessar(frame)
                locais = proc.segmentar(rgb)

                if len(locais) == 0:
                    self.lbl_status.configure(text="Nenhum rosto encontrado
na câmara", text_color="red")
                elif len(locais) > 1:
                    self.lbl_status.configure(text="Apenas UMA pessoa na
tela para o cadastro", text_color="red")
                else:
                    top, right, bottom, left =
proc.caixa_no_frame_original(locais[0])
                    cv2.rectangle(frame, (left, top), (right, bottom), (0,
255, 0), 2)

                    agora = time.time()
                    if agora - ultimo >= config.INTERVALO_AMOSTRA_S:
                        enc = proc.extrair_caracteristicas(rgb, locais)[0]
```

```
                        amostras.append(enc)
                        ultimo = agora

                    self.lbl_status.configure(
                        text=f"Amostras capturadas:
{len(amostras)}/{config.AMOSTRAS_CADASTRO}",
                        text_color="cyan",
                    )

                self._atualizar_video(frame)
                time.sleep(0.03)

            if len(amostras) == config.AMOSTRAS_CADASTRO:
                banco.cadastrar_usuario(nome, cargo, nivel, amostras)
                banco.registrar_log(self.controller.usuario_logado,
"CADASTRO_UTILIZADOR", f"Utilizador '{nome}' cadastrado com sucesso (Nível
{nivel})")
                self.lbl_status.configure(text=f"✅ Utilizador '{nome}'
cadastrado com sucesso!", text_color="green")
                messagebox.showinfo("Sucesso", f"Utilizador {nome}
cadastrado com sucesso!")
                self.ent_nome.delete(0, 'end')
                self.ent_cargo.delete(0, 'end')
        except Exception as e:
            messagebox.showerror("Erro ao Cadastrar", str(e))
        finally:
            if self.cap:
                self.cap.release()
            self.rodando = False
            self.btn_capturar.configure(state="normal")
            self.after(0, self.resetar_visor)
```

A TelaCadastro, exclusiva do ROOT, reúne nome, cargo e nível. O método iniciar_captura valida o nome, desabilita o botão e inicia _loop_captura em thread, que reproduz o cadastro.py: coleta as AMOSTRAS_CADASTRO (10) com intervalo mínimo INTERVALO_AMOSTRA_S, somente com exatamente um rosto, e mostra o progresso. Concluída a coleta, grava com “banco.cadastrar_usuario”, registra CADASTRO_UTILIZADOR no log e limpa o formulário. Erros, como nome duplicado (ValueError do banco), são mostrados em caixa de diálogo, e o “finally” libera a câmera e reabilita o botão.

1.8.6 telas de usuários e de rastreio

```
    def ao_exibir(self):
        for row in self.tree.get_children():
            self.tree.delete(row)
        for u in banco.listar_usuarios():
            self.tree.insert("", "end", values=u)

    def remover(self):
        item = self.tree.selection()
        if not item:
            messagebox.showwarning("Aviso", "Selecione um utilizador para
remover.")
            return

        dados_utilizador = self.tree.item(item[0])["values"]
        uid = dados_utilizador[0]
        nome_removido = dados_utilizador[1]

        if messagebox.askyesno("Confirmar", f"Deseja remover o utilizador ID
{uid} ({nome_removido})?"):
            banco.remover_usuario(uid)
            banco.registrar_log(self.controller.usuario_logado,
"EXCLUSAO_UTILIZADOR", f"Removeu o utilizador '{nome_removido}' (ID:
{uid})")
            self.ao_exibir()

    def ao_exibir(self):
        for row in self.tree_logs.get_children():
            self.tree_logs.delete(row)

        logs = banco.listar_logs()
        for log in logs:
            self.tree_logs.insert("", "end", values=log)

    def exportar_logs(self):
        logs = banco.listar_logs()
        if not logs:
            messagebox.showwarning("Aviso", "Não existem logs registados
para exportar.")
            return

        caminho = filedialog.asksaveasfilename(
            defaultextension=".csv",
            filetypes=[("Ficheiro CSV", "*.csv"), ("Ficheiro de Texto",
"*.txt")],
            title="Guardar Relatório de Rastreio"
        )
        if not caminho:
```

```
            return

        try:
            with open(caminho, "w", encoding="utf-8") as f:
                f.write("ID;Utilizador;Acao;Detalhes;Data_Hora\n")
                for log in logs:

f.write(f"{log[0]};{log[1]};{log[2]};{log[3]};{log[4]}\n")

            banco.registrar_log(self.controller.usuario_logado,
"EXPORTAR_LOGS", f"Exportou {len(logs)} registos de auditoria")
            messagebox.showinfo("Sucesso", f"Relatório de rastreio exportado
com sucesso para:\n{caminho}")
            self.ao_exibir()
        except Exception as e:
            messagebox.showerror("Erro", f"Não foi possível exportar
o ficheiro: {str(e)}")
```

A TelaUsuarios lista os usuários em uma tabela “ttk.Treeview” (ID, nome, cargo, nível e amostras), recarregada por ao_exibir. O método remover pede confirmação (“askyesno”), remove o usuário (e, por cascata, seus vetores) e registra EXCLUSAO_UTILIZADOR. A TelaRastreio, exclusiva do ROOT, exibe a trilha de auditoria; o método exportar_logs abre a janela de salvar, grava um CSV em UTF-8 com campos separados por ponto e vírgula (formato que o Excel abre diretamente), registra EXPORTAR_LOGS e mostra erros em caixa de diálogo.

1.8.7 tela de arquivos do cofre

```
    def subir_arquivo(self):
        caminho = filedialog.askopenfilename(title="Selecionar Ficheiro para
Upload")
        if not caminho:
            return

        nome_arquivo = os.path.basename(caminho)
        nivel_minimo = int(self.combo_nivel_arquivo.get().split(" ")[0])

        try:
            banco.salvar_arquivo_banco(nome_arquivo, caminho, nivel_minimo)
```

```
            banco.registrar_log(self.controller.usuario_logado,
"UPLOAD_FICHEIRO", f"Subiu o ficheiro '{nome_arquivo}' (Nível mín:
{nivel_minimo})")
            messagebox.showinfo("Sucesso", f"Ficheiro '{nome_arquivo}'
enviado com sucesso!")
            self.ao_exibir()
        except Exception as e:
            messagebox.showerror("Erro", f"Não foi possível salvar o
ficheiro: {str(e)}")

    def abrir_ficheiro_selecionado(self, event=None):
        selecao = self.tree_arq.selection()
        if not selecao:
            messagebox.showwarning("Aviso", "Selecione um ficheiro na lista
para abrir.")
            return

        item_id = self.tree_arq.item(selecao[0])["values"][0]
        nome_arquivo = self.tree_arq.item(selecao[0])["values"][1]
        caminho_real = self.caminhos_arquivos.get(item_id)

        if caminho_real and os.path.exists(caminho_real):
            try:
                banco.registrar_log(self.controller.usuario_logado,
"ABRIR_FICHEIRO", f"Abriu o ficheiro '{nome_arquivo}'")
                os.startfile(caminho_real)
            except Exception as e:
                messagebox.showerror("Erro", f"Não foi possível abrir o
ficheiro: {str(e)}")
        else:
            messagebox.showerror("Erro", "O ficheiro físico não foi
encontrado no caminho original.")

    def ao_exibir(self):
        for row in self.tree_arq.get_children():
            self.tree_arq.delete(row)

        self.caminhos_arquivos.clear()
        nivel_atual = self.controller.nivel_usuario
        arquivos = banco.listar_arquivos_por_nivel(nivel_atual)

        for arq in arquivos:
            uid, nome, caminho, nivel_min, data = arq
            self.caminhos_arquivos[uid] = caminho
            self.tree_arq.insert("", "end", values=(uid, nome, nivel_min,
data))

if __name__ == "__main__":
    app = CofreApp()
```

A TelaRelatorios implementa o armazenamento de arquivos por nível. O ROOT envia arquivos (subir_arquivo grava o caminho e o nível mínimo, registrando UPLOAD_FICHEIRO) e exclui registros, sem apagar o arquivo original. Todos veem apenas os arquivos permitidos, pois ao_exibir consulta “banco.listar_arquivos_por_nivel” com o nível do usuário (o ROOT, de nível 3, vê tudo). O duplo clique chama abrir_ficheiro_selecionado, que localiza o caminho no dicionário caminhos_arquivos, registra ABRIR_FICHEIRO e abre o arquivo com “os.startfile”; se o arquivo não existir, exibe um erro.

Todas as ações relevantes (login, autenticação do cofre, cadastro, exclusão, exportação e manuseio de arquivos) deixam registro em logs_sistema, consultável apenas pelo administrador. As falhas previsíveis geram avisos em “messagebox”, e os laços de vídeo usam “try … finally” para sempre liberar a câmera.

1.9 MÓDULO MAIN.PY

```
"""
Ponto de entrada principal do sistema do cofre biométrico com Interface
Gráfica
"""
from gui import CofreApp

if __name__ == "__main__":
   app = CofreApp()
```

O main.py é o ponto de entrada: importa a classe CofreApp e, sob “if __name__ == "__main__"”, cria a janela e inicia o “mainloop()”, que mantém a interface ativa até o fechamento. Com o arquivo enxuto, a forma de apresentar o sistema pode mudar sem alterar os módulos de reconhecimento, banco e relatórios.